



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura:
Estructura de Datos y Algoritmos II

Proyecto final:
Software para la visualización y estudio de algoritmos y estructuras de datos
-Backend-

Documentación Técnica del Módulo de Algoritmos

Análisis, Diseño e Implementación de Algoritmos en el Backend

Carpeta documentada:
app/algoritmos/

Equipo de Desarrollo de Backend

21 de mayo de 2026

1. Módulo de Algoritmos

La carpeta `algoritmos` contiene las implementaciones principales de estructuras de datos, algoritmos de búsqueda, ordenamiento, grafos, manejo de archivos y procesamiento paralelo. Su función dentro del sistema es concentrar la lógica computacional que posteriormente puede ser utilizada por la API y representada visualmente en el front-end.

Este módulo no está diseñado para mostrarse directamente al usuario final, sino para proporcionar las operaciones que serán consumidas desde la interfaz. Por ello, la documentación se enfoca en describir brevemente qué hace cada archivo y cómo podría representarse en la parte visual del sistema.

1.1. Carpeta arboles

La carpeta `arboles` contiene algoritmos relacionados con estructuras jerárquicas y conversión de expresiones matemáticas. Incluye árboles binarios, árboles B, árboles B+ y conversión entre notaciones infija, prefija y sufija.

Archivo	Descripción breve	Uso en front-end
<code>binary_tree.py</code>	Implementa un árbol binario de búsqueda con inserción, búsqueda, eliminación y recorridos.	Permite visualizar un árbol, mostrar recorridos como inorden, preorden, postorden y nivel-orden.
<code>btree.py</code>	Implementa un Árbol B de orden 3, útil para almacenar claves en nodos con varios hijos.	Puede mostrarse como una estructura por niveles, representando cada nodo como una caja con claves.
<code>bplustree.py</code>	Implementa un Árbol B+ donde los datos se almacenan en hojas enlazadas.	Puede visualizarse mostrando hojas conectadas y valores ordenados secuencialmente.
<code>notation.py</code>	Convierte expresiones infijas a notación sufija o prefija.	Puede usarse en una vista donde el usuario escriba una expresión y vea la conversión.
<code>router.py</code>	Expone endpoints para trabajar con árboles y notaciones.	Permite que el front-end mande datos y reciba resultados en formato JSON.

Cuadro 1: Resumen de archivos de la carpeta arboles

En la interfaz, esta sección puede funcionar como un visualizador de árboles. El usuario ingresaría valores, seleccionaría el tipo de árbol u operación, y el sistema mostraría el resultado mediante recorridos, estructuras por niveles o conversiones de expresiones.

1.2. Carpeta archivos

La carpeta `archivos` contiene funciones para guardar archivos en el backend y consultar información básica de ellos.

Archivo	Descripción breve	Uso en front-end
<code>organizacion.py</code>	Crea y administra la carpeta donde se guardan los archivos subidos.	Permite que los archivos cargados desde la interfaz tengan una ubicación definida en el backend.
<code>operaciones.py</code>	Guarda archivos recibidos y obtiene información como nombre, tamaño y fecha de creación.	Puede usarse para mostrar una tabla o tarjeta con los datos del archivo subido.
<code>__init__.py</code>	Exporta las funciones principales del módulo.	Facilita que otros módulos del backend usen estas funciones.

Cuadro 2: Resumen de archivos de la carpeta archivos

En el front-end, esta parte puede representarse con un formulario de carga de archivos. El usuario selecciona un archivo, lo sube al sistema y posteriormente se muestran sus datos principales.

1.3. Carpeta busqueda

La carpeta `busqueda` contiene algoritmos para localizar un valor dentro de un arreglo.

Archivo	Descripción breve	Uso en front-end
<code>busqueda_lineal.py</code>	Recorre el arreglo elemento por elemento hasta encontrar el valor objetivo.	Puede mostrarse como una búsqueda paso a paso resaltando cada comparación.
<code>busqueda_binaria.py</code>	Busca un valor dividiendo el arreglo ordenado en mitades.	Puede visualizarse reduciendo el rango de búsqueda en cada paso.
<code>hash.py</code>	Usa una tabla hash para buscar valores de forma eficiente.	Puede mostrarse como una tabla clave-valor indicando si el elemento fue encontrado.

Cuadro 3: Resumen de archivos de la carpeta busqueda

En la interfaz, el usuario podría ingresar un arreglo, seleccionar un algoritmo de búsqueda y escribir el valor a buscar. El sistema mostraría si el valor fue encontrado, su índice, número de comparaciones o colisiones y complejidad aproximada.

1.4. Carpeta grafos

La carpeta **grafos** contiene funciones para representar grafos y recorrerlos mediante BFS y DFS.

Archivo	Descripción breve	Uso en front-end
<code>representacion.py</code>	Genera matriz y lista de adyacencia a partir de nodos y aristas.	Permite mostrar el grafo como tabla, matriz o diagrama visual.
<code>bfs.py</code>	Implementa búsqueda en anchura usando una cola.	Puede visualizarse recorriendo el grafo por niveles.
<code>dfs.py</code>	Implementa búsqueda en profundidad usando recursividad.	Puede visualizarse siguiendo un camino profundo antes de retroceder.

Cuadro 4: Resumen de archivos de la carpeta grafos

En el front-end, esta sección puede permitir al usuario crear un grafo ingresando nodos y aristas. Después, podrá seleccionar BFS o DFS y observar el orden en que se visitan los nodos.

1.5. Carpeta ordenamiento

La carpeta **ordenamiento** contiene algoritmos para ordenar arreglos numéricos. Algunos de estos algoritmos registran pasos intermedios, lo cual permite crear animaciones o visualizaciones didácticas en el front-end.

Archivo	Descripción breve	Uso en front-end
<code>sorting_algorithms.py</code>	Implementa Bubble Sort, Heap Sort y Quick Sort con pasos intermedios.	Permite animar comparaciones, intercambios, pivotes y estados del arreglo durante el ordenamiento.
<code>sorting_router.py</code>	Expone endpoints para ejecutar Bubble Sort, Heap Sort y Quick Sort mediante FastAPI. Recibe un arreglo de enteros y devuelve el resultado generado por cada algoritmo.	Permite que el front-end mande un arreglo al backend y reciba la información necesaria para mostrar el ordenamiento visualmente.
<code>sorting_schemas.py</code>	Define modelos de entrada y salida para estructurar los datos de ordenamiento.	Ayuda a estandarizar la información que el front-end envía y recibe.
<code>counting_sort.py</code>	Ordena enteros contando ocurrencias dentro de un rango.	Puede mostrarse con una tabla de frecuencias y el arreglo final ordenado.
<code>merge_sort.py</code>	Ordena dividiendo el arreglo en mitades y mezclándolas ordenadamente.	Puede visualizarse como división y combinación de subarreglos.
<code>radix_sort.py</code>	Ordena números procesando sus dígitos de menor a mayor posición.	Puede mostrarse indicando el dígito usado en cada etapa del ordenamiento.

Cuadro 5: Resumen de archivos de la carpeta ordenamiento

En el front-end, esta parte puede funcionar como un visualizador de algoritmos de ordenamiento. El usuario ingresa un arreglo, selecciona el algoritmo y observa el proceso mediante barras, listas o tablas de pasos.

Los endpoints principales expuestos para esta sección son:

- `/bubble-sort`: ejecuta Bubble Sort.
- `/heap-sort`: ejecuta Heap Sort.
- `/quick-sort`: ejecuta Quick Sort.

Estos endpoints reciben una estructura con el siguiente formato:

```
{
  "arreglo": [64, 34, 25, 12, 22, 11, 90]
}
```

La respuesta puede utilizarse para mostrar el arreglo ordenado, el número de pasos realizados y los estados intermedios del proceso, dependiendo del algoritmo seleccionado.

1.6. Carpeta paralelos

La carpeta `paralelos` contiene funciones relacionadas con procesamiento paralelo, comparación de rendimiento y teoría de paralelismo.

Archivo	Descripción breve	Uso en front-end
<code>paralelos.py</code>	Implementa búsqueda de primos secuencial y paralela, cálculos por rango, comparación de rendimiento y datos teóricos sobre paralelismo.	Puede mostrarse como una sección de pruebas de rendimiento con tiempos, speedup, eficiencia y número de procesos.

Cuadro 6: Resumen de archivos de la carpeta `paralelos`

En el front-end, este módulo puede representarse con formularios para ejecutar cálculos secuenciales o paralelos. También puede incluir gráficas comparativas de tiempo, speedup y eficiencia.

2. Integración General con Front-End

La integración visual del módulo `algoritmos` se basa en permitir que el usuario interactúe con cada algoritmo mediante formularios y componentes gráficos. El front-end no debe implementar directamente la lógica interna de los algoritmos, sino enviar los datos al backend y mostrar los resultados recibidos.

De forma general, el flujo sería:

1. El usuario ingresa datos desde la interfaz.
2. El front-end valida la información básica.
3. Se envía una petición al backend.
4. El backend ejecuta el algoritmo correspondiente.
5. El resultado se devuelve en formato JSON.
6. El front-end muestra el resultado de forma visual.

Cada módulo puede tener una representación visual diferente:

- **Árboles:** nodos conectados, recorridos y niveles.
- **Archivos:** carga de archivos y tarjetas con metadatos.

- **Búsqueda:** arreglo con elementos resaltados durante la búsqueda.
- **Grafos:** nodos, aristas y recorridos BFS/DFS.
- **Ordenamiento:** barras animadas, comparaciones e intercambios.
- **Paralelos:** gráficas de tiempo, speedup y eficiencia.

Esta integración permite que la documentación no solo explique el código, sino que también sirva como guía para que el equipo de front-end implemente las vistas correspondientes.

3. Conclusión

La carpeta **algoritmos** concentra la lógica principal del sistema relacionada con estructuras de datos y técnicas algorítmicas. Aunque cada submódulo resuelve un problema diferente, todos comparten el mismo propósito: proporcionar operaciones que puedan ser consumidas por el backend y representadas visualmente desde el front-end.

Esta documentación sintetizada permite identificar rápidamente qué contiene cada archivo, cuál es su función principal y cómo puede utilizarse dentro de la interfaz del sistema.